

SubQuery

A subquery is a query nested inside another statement such as SELECT, INSERT, UPDATE, or DELETE.

Example:

-- Inner query will execute first then the outer query will execute

Select * from EmployeeDetails where Empid in (Select Empid from EmpSalary)

Select * from EmployeeDetails where Empid not in (Select Empid from EmpSalary)

Nesting subquery: A subquery can be nested within another subquery. SQL Server supports up to 32 levels of nesting.

```
Select EmpID, EmpName, DeptID
from EmployeeDetails
Where EmpID in (Select EmpID from EmpSalary Where DeptID in (Select DeptID
from Department ))
```

Co-related subquery: A correlated subquery is a subquery that uses the values of the outer query. In other words, the correlated subquery depends on the outer query for its values.

Because of this dependency, a correlated subquery cannot be executed independently as a simple subquery.

Example:

```
Select EmpID, EmpName, DeptID
from EmployeeDetails e1
Where EmpID in(
Select EmpID from EmpSalary e2
where e1.EmpID = e2.EmpID
)
```

Derived table: A derived table is a subquery nested within a FROM clause. Because of being in a FROM clause, the subquery's result set can be used similarly to a SQL Server table. The subquery in the FROM clause must have a name. One reason for including a derived table in an outer query is to simplify the outer query. This is because you can bury the complexity of a subquery in a derived table in a FROM clause and refer to subquery result column values in the derived table. Because you can treat derived tables like regular SQL Server tables, you can join two or more derived tables. Also, you can nest queries referencing a derived table inside of one another.

```
Select * from (Select dense_rank() over (order by empSalary desc) As RNK,* from empSalary ) k Where k.RNK =3
```

```
Select * from (Select dense_rank() over (order by empSalary desc) As RNK,* from empSalary ) k
```

-- Filter duplicate value

```
Select * from EmpSalary order by 1
```

```
Select distinct * from EmpSalary order by 1
```

-- Find the Distinct value without using Distinct

```
select * from ( Select *, row_number() over (partition by EmpID, EmpName, EmpSalary, DeptID order by EmpID desc) As RNK from EmpSalary) k  
Where k.RNK = 1
```

-- Find the values which are duplicate

```
select * from ( Select *, row_number() over (partition by EmpID, EmpName, EmpSalary, DeptID order by EmpID desc) As RNK from EmpSalary) k  
Where k.RNK>1
```